Matematyczne metody optymalizacji
Podgrupa 2

Michał Dziwisz 259154
Michał Pryczek 259162
Jakub Sońta 259166

1. Sformułowanie problemu (Funkcja celu)

Celem zadania jest znalezienie punktu na wykresie funkcji $f(x) = 7 - x^{5/2}$, który minimalizuje kwadrat odległości od punktu P = (4,5; 8).

Punkt na wykresie funkcji ma współrzędne (x, f(x)). Kwadrat odległości D między dowolnym punktem (x, y) a punktem (x_0, y_0) wyraża się wzorem:

$$D = (x - x_0)^2 + (y - y_0)^2$$

Podstawiając dane z zadania:

- x_0 = 4,5
- y_0 = 8
- y = $7 - x^{5/2}$

Funkcja celu g(x):

$$g(x) = (x - 4{,}5)^2 + \left(7 - x^{5/2} - 8\right)^2$$

$$g(x) = (x - 4{,}5)^2 + (-1 - x^{5/2})^2$$

$$g(x) = (x - 4{,}5)^2 + (1 + x^{5/2})^2$$

2. Ograniczenia

Dziedzina funkcji $f(x) = 7 - x^{5/2}$ należy do zbioru $x \in \langle 0, +\infty)$, ponieważ potęgi 5/2 implikuje operację pierwiastkowania stopnia parzystego, która w zbiorze liczb rzeczywistych jest zdefiniowana wyłącznie dla argumentów większych lub równych zero.

3. Analityczna metoda rozwiązania

Aby zminimalizować funkcję celu g(x), należy wyznaczyć jej pochodną, przyrównać ją do zera (warunek konieczny istnienia ekstremum) oraz sprawdzić drugą pochodną (warunek wystarczający).

- Wyznaczenie pochodnej g'(x):

$$g'(x) = -9 + 2x + 5x^{3/2} + 5x^4$$

- Przyrównanie do zera:

$$5x^4 + 5x^{3/2} + 2x - 9 = 0$$

Używając funkcji pomocniczej $5t^8 + 5t^5 + 2t^2 - 9 = 0$ zakładając, że dla $x = t^2 (t \geq 0)$ wykorzystując twierdzenie Bézouta wynika, że pierwiastki wymierne (jeśli istnieją) muszą mieć postać:

$$\pm 1, \pm 3, \pm 9, \pm \frac{1}{5}, \pm \frac{3}{5}, \pm \frac{9}{5}.$$

Sprawdzanie wyników dla $f(t) = 5t^8 + 5t^5 + 2t^2 - 9$:

- $f(1) = 5 + 5 + 2 - 9 = 3 \neq 0$
- $f(-1) = 5 - 5 + 2 - 9 = -7 \neq 0$
- $f(3) = 5 \cdot 6561 + 5 \cdot 243 + 2 \cdot 9 - 9 > 0$
- $f(-3) = 5 \cdot 6561 - 5 \cdot 243 + 18 - 9 > 0$
- $f(9) > 0$
- $f(-9) > 0$
- $f\left(\frac{1}{5}\right) = 5 \cdot \frac{1}{390625} + 5 \cdot \frac{1}{3125} + 2 \cdot \frac{1}{25} - 9 < 0$
- $f\left(-\frac{1}{5}\right) < 0$
- $f\left(\frac{3}{5}\right) = 5 \cdot \frac{6561}{390625} + 5 \cdot \frac{243}{3125} + 2 \cdot \frac{9}{25} - 9 \neq 0$
- $f\left(-\frac{3}{5}\right) \neq 0$
- $f\left(\frac{9}{5}\right) > 0$
- $f\left(-\frac{9}{5}\right) > 0$

Ponieważ nie posiada pierwiastków wymiernych nie istnieje analityczna metoda rozwiązania tego równania. Dlatego w programie matlab można użyć tylko funkcji numerycznych takich jak np. vpasolve.

3.1 Program matlab

```
syms x real

Px = 4.5;
Py = 8;
epsilon = 1e-6;
f = 7 - x^(5/2);
D = (x - Px)^2 + (f - Py)^2;

fprintf('1. Funkcja celu (Sformułowana symbolicznie):\n');
pretty(D)

d_sym = diff(D, x);

fprintf('\n2. Pochodna funkcji celu D''(x):\n');
pretty(simplify(d_sym))

rozwiazanie_vpasolve = vpasolve(d_sym == 0, x, [0 Inf]);
x_opt_vpasolve = double(rozwiazanie_vpasolve);

% Obliczenie wartości y oraz minimalnej odległości
```

```
y_opt_vpasolve = double(subs(f, x, x_opt_vpasolve));
min_dist_sq_vpasolve = double(subs(D, x, x_opt_vpasolve));
min_dist_vpasolve = sqrt(min_dist_sq_vpasolve);

fprintf('Znalezione x (vpasolve): %.6f\n', x_opt_vpasolve);
fprintf('Znalezione y (vpasolve): %.6f\n', y_opt_vpasolve);
fprintf('Minimalna odległość: %.6f\n\n', min_dist_vpasolve);
```

```
>> proj()
1. Funkcja celu (Sformułowana symbolicznie):
/     9 \2     5/2     2
| x - - |  + (x    + 1)
\     2 /


2. Pochodna funkcji celu D'(x):
         4      3/2
2 x + 5 x  + 5 x    - 9

Znalezione x (vpasolve): 0.885000
Znalezione y (vpasolve): 6.263186
Minimalna odległość: 4.010580
```

4. Numeryczna metoda rozwiązania

```
f_num = @(x) 7 - x.^(5/2);
D_num = @(x) (x - Px).^2 + (f_num(x) - Py).^2;

options = optimset('Display','iter','TolX',epsilon);

fprintf('--- ROZWIĄZANIE NUMERYCZNE (fminbnd) ---\n');
[x_opt_num, fval_num, exitflag, output] = fminbnd(D_num, 0, 5, options);

y_opt_num = f_num(x_opt_num);
min_dist_num = sqrt(fval_num);

fprintf('\nZnalezione x (numerycznie): %.6f\n', x_opt_num);
fprintf('Znalezione y (numerycznie): %.6f\n', y_opt_num);
fprintf('Minimalna odległość: %.6f\n', min_dist_num);
fprintf('Liczba iteracji: %d\n', output.iterations);
```

```
--- ROZWIĄZNIE NUMERYCZNE (fminbnd) ---

 Func-count     x          f(x)         Procedure
    1        1.90983      43.1985        initial
    2        3.09017      318.341        golden
    3        1.18034      17.3384        golden
    4        1.37382      20.0913        parabolic
    5         1.0974      16.6924        parabolic
    6        0.67823      16.5071        golden
    7       0.872709      16.0865        parabolic
    8       0.868749      16.0877        parabolic
    9       0.887385      16.0848        parabolic
```

```
   10       0.967604      16.1679        golden
   11       0.885049      16.0847        parabolic
   12       0.884989      16.0847        parabolic
   13          0.885      16.0847        parabolic
   14          0.885      16.0847        parabolic
   15          0.885      16.0847        parabolic

Optimization terminated:
 the current x satisfies the termination criteria using OPTIONS.TolX of 1.000000e-
06


Znalezione x (numerycznie): 0.885000
Znalezione y (numerycznie): 6.263185
Minimalna odległość: 4.010580
Liczba iteracji: 14
```

5. Metoda bisekcji

```
fprintf('\n--- METODA BISEKCJI ---\n');
a = 0;
b = 5;
iter = 0;

fileID = fopen('wyniki_bisekcja.txt', 'w');
fprintf(fileID, 'Iteracja\t x_srodek\t Blad_bezwzgledny\n');

fprintf('Iter\t x_approx\t Blad\n');

while (b - a) >= (2 * epsilon)
    iter = iter + 1;

    x1 = a + (b - a)/4;
    xm = (a + b)/2;
    x2 = b - (b - a)/4;

    fx1 = D_num(x1);
    fxm = D_num(xm);
    fx2 = D_num(x2);

    if fx1 < fxm
        b = xm;
    elseif fx2 < fxm
        a = xm;
    else
        a = x1;
        b = x2;
    end
    x_approx = (a + b)/2;

    blad = abs(x_opt_vpasolve - x_approx);

    % Zapis do pliku i wyświetlenie w konsoli
    fprintf(fileID, '%d\t %.8f\t %.8f\n', iter, x_approx, blad);
    fprintf('%d\t %.6f\t %.8f\n', iter, x_approx, blad);
```

```
end

x_bisekcja = (a + b)/2;
y_bisekcja = f_num(x_bisekcja);
dist_bisekcja = sqrt(D_num(x_bisekcja));

fprintf('\n--- PODSUMOWANIE BISEKCJI ---\n');
fprintf('Wynik zapisano w pliku: wyniki_bisekcja.txt\n');
fprintf('Znalezione x (bisekcja): %.6f\n', x_bisekcja);
fprintf('Liczba iteracji: %d\n', iter);
```

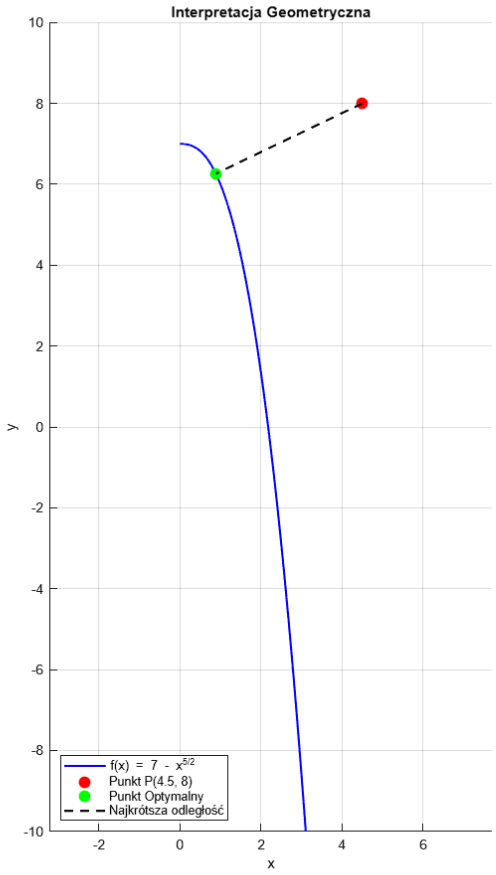
```
--- METODA BISEKCJI ---
Iter    x_approx     Blad
1       1.250000     0.36500004
2       0.625000     0.25999996
3       0.937500     0.05250004
4       0.937500     0.05250004
5       0.859375     0.02562496
6       0.898438     0.01343754
7       0.878906     0.00609371
8       0.888672     0.00367191
9       0.883789     0.00121090
10      0.883789     0.00121090
11      0.885010     0.00000981
12      0.885010     0.00000981
13      0.885010     0.00000981
14      0.885010     0.00000981
15      0.885010     0.00000981
16      0.885010     0.00000981
17      0.884991     0.00000927
18      0.885000     0.00000027
19      0.885000     0.00000027
20      0.885000     0.00000027
21      0.885000     0.00000027
22      0.885000     0.00000027

--- PODSUMOWANIE BISEKCJI ---
Wynik zapisano w pliku: wyniki_bisekcja.txt
Znalezione x (bisekcja): 0.885000
Liczba iteracji: 22
```

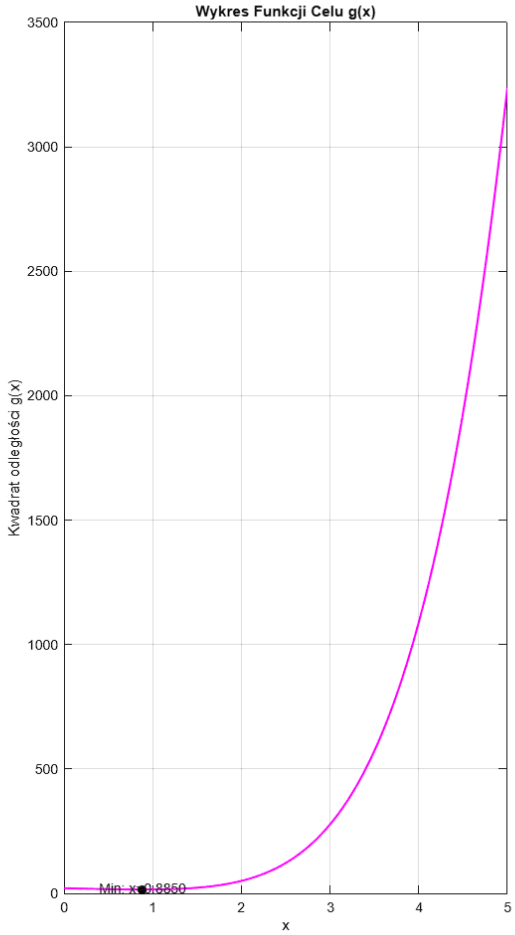
6. Graficzna interpretacja rozwiązania

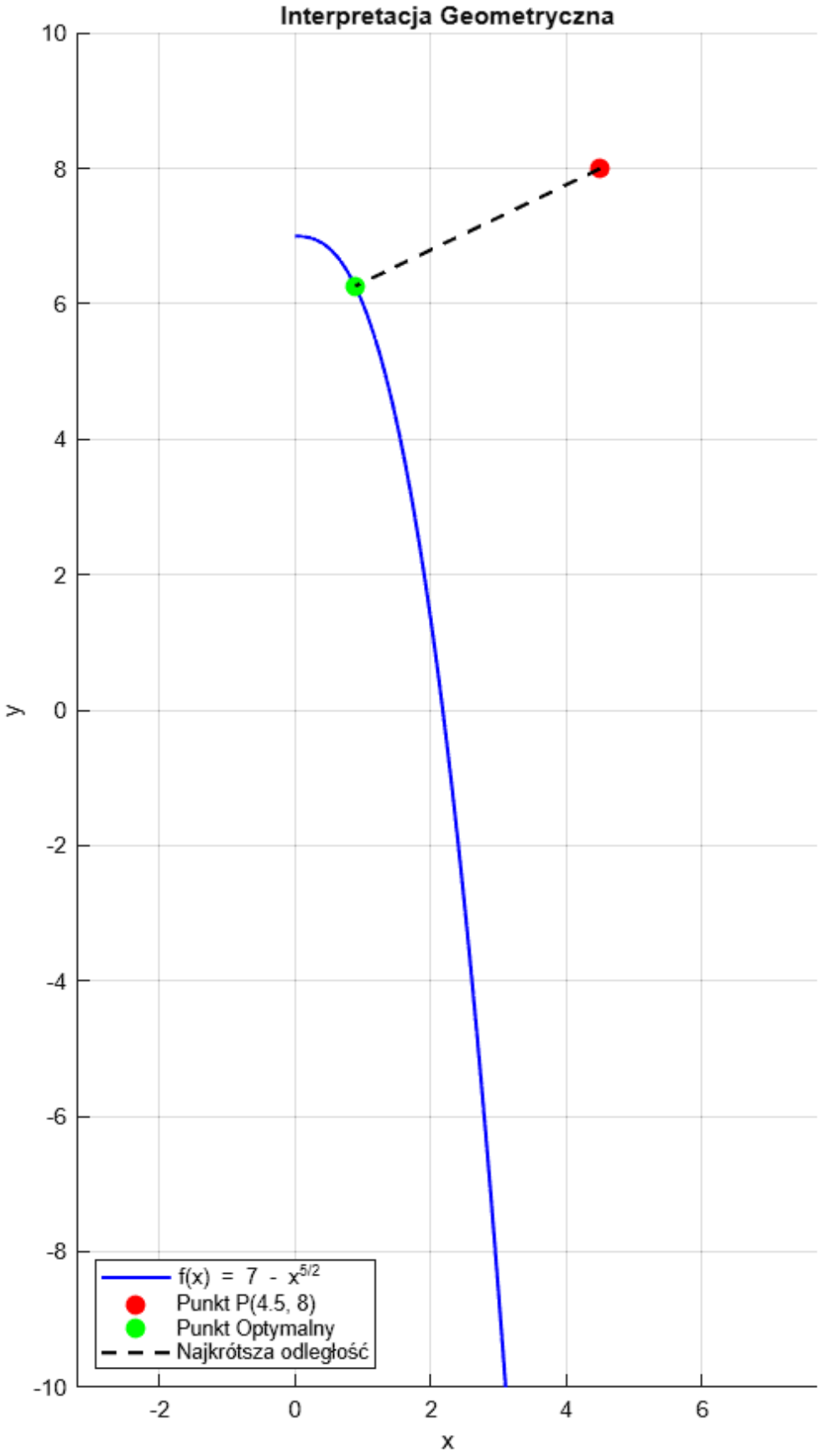
```
figure('Name', 'Projekt Optymalizacja', 'NumberTitle', 'off', 'Color', 'w');

% Wykres 1: Geometria problemu (Krzywa i najkrótszy odcinek)
subplot(1,2,1);
hold on; grid on; axis equal;
x_plot = 0:0.01:5;
y_plot = f_num(x_plot);
```

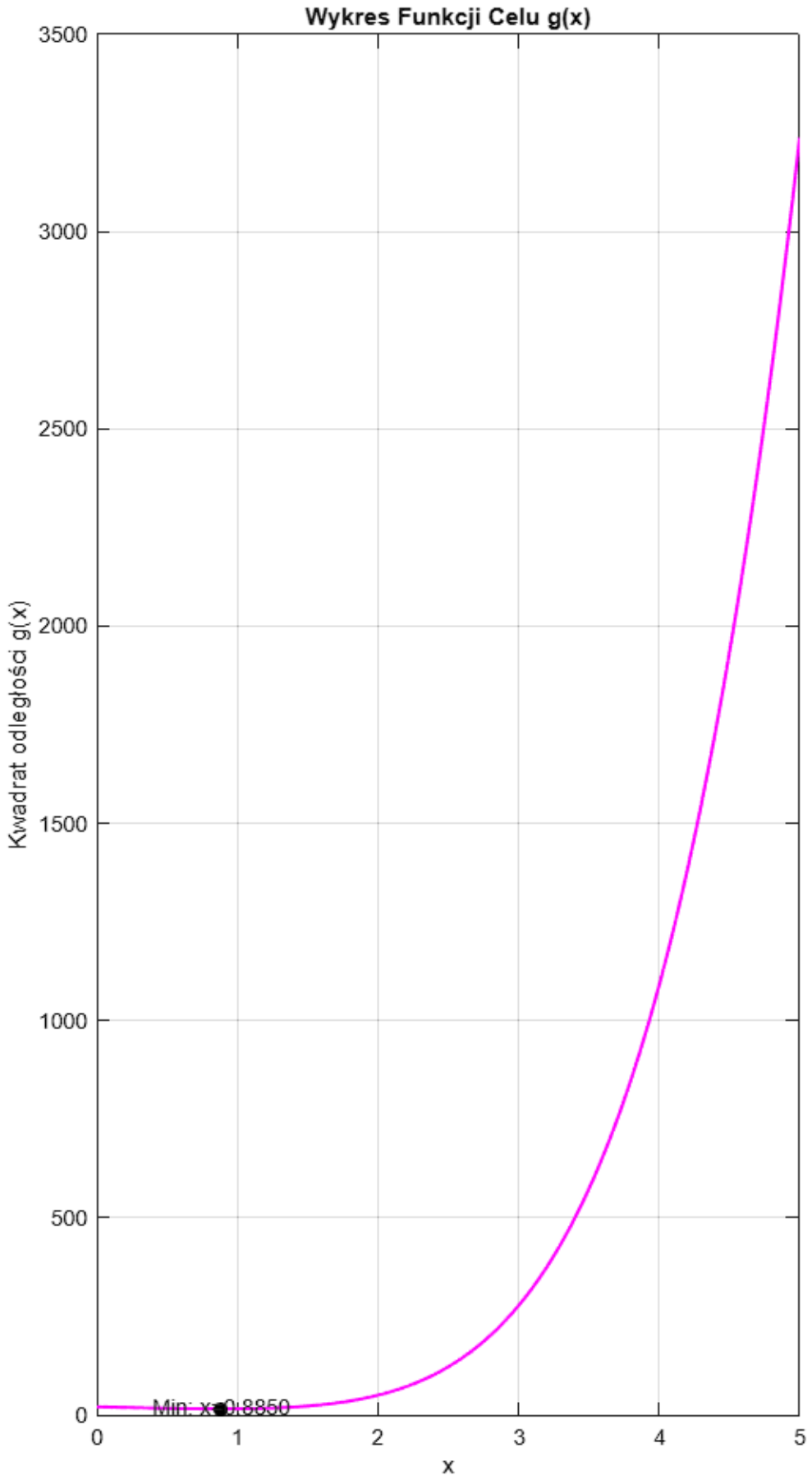
```
plot(x_plot, y_plot, 'b-', 'LineWidth', 1.5, 'DisplayName', 'f(x) = 7 - x^{5/2}');
plot(Px, Py, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 8, 'DisplayName', 'Punkt
P(4.5, 8)');
plot(x_opt_num, y_opt_num, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 8,
'DisplayName', 'Punkt Optymalny');
plot([Px x_opt_num], [Py y_opt_num], 'k--', 'LineWidth', 1.5, 'DisplayName',
'Najkrótsza odległość');

xlabel('x'); ylabel('y');
title('Interpretacja Geometryczna');
legend('Location', 'southwest');
ylim([-10 10]);

% Wykres 2: Funkcja celu (Parabola minimalizacji)
subplot(1,2,2);
g_plot = D_num(x_plot);
plot(x_plot, g_plot, 'm-', 'LineWidth', 1.5);
hold on; grid on;
plot(x_opt_num, fval_num, 'ko', 'MarkerFaceColor', 'k');

xlabel('x'); ylabel('Kwadrat odległości g(x)');
title('Wykres Funkcji Celu g(x)');
text(x_opt_num, fval_num+5, sprintf('Min: x=%.4f', x_opt_num),
'HorizontalAlignment', 'center');

sgtitle('Minimalizacja odległości punktu od funkcji');
```

Minimalizacja odległości punktu od funkcji






Interpretacja Geometryczna
10
8
6
4
2
0
-2
-4
-6
-8
-10
y
-2
0
2
4
6
x
f(x) = 7 - x^5/2
Punkt P(4.5, 8)
Punkt Optymalny
Najkrótsza odległość


Wykres Funkcji Celu g(x)
Kwadrat odległości g(x)
x
3500
3000
2500
2000
1500
1000
500
0
0
1
2
3
4
5
Min: x=0.8850

7. Porównanie oraz dyskusja otrzymanych wyników

```
==============================================
     ZESTAWIENIE REZULTATÓW KOŃCOWYCH
==============================================
Metoda          | x*           | Min. Odleglosc
----------------------------------------------
Matlab vpasolve | 0.885000     | 4.010580
Matlab fminbnd  | 0.885000     | 4.010580
Własna Bisekcja | 0.885000     | 4.010580
----------------------------------------------
```

Dla danej funkcji celu nie jest możliwe analityczne wyznaczenie punktu na wykresie funkcji $\mathrm{g(x)}$, który minimalizuje odległość od punktu P. Wszystkie metody osiągnęły te same wyniki dla dokładności ε=10-6. Istotną różnicę pomiędzy zastosowanymi metodami numerycznymi widać w szybkości zbieżności algorytmów:

- **Funkcja fminbnd (MATLAB):** Wymagała **14 iteracji**. Funkcja ta wykorzystuje hybrydę metody złotego podziału i interpolacji parabolicznej. Gdy algorytm znajduje się blisko minimum, aproksymuje funkcję parabolą, co pozwala na bardzo szybką (często kwadratową) zbieżność.
- **Własna metoda bisekcji:** Wymagała **22 iteracji**. Jest to metoda o zbieżności liniowej. W każdym kroku przedział poszukiwań jest redukowany o stały czynnik. Jest to metoda bardzo stabilna i gwarantująca znalezienie wyniku (o ile przedział początkowy jest poprawny), ale wolniejsza od metod wykorzystujących pochodne lub interpolacje.

8. Minimalizacja powierzchni całkowitej walca przy stałej objętości

Celem zadania było znalezienie takich wymiarów walca (promienia r i wysokości h), które minimalizują jego pole powierzchni całkowitej P, przy zachowaniu stałej objętości V.

Dane wejściowe:

- Objętość $V = 0.885000\ m3$ (wartość wynikająca z rozwiązania poprzedniego zadania).
- Dokładność obliczeń $\varepsilon = 10^{-6}$.

Wzór na objętość walca:

$$V = \pi r^2 h \Rightarrow h = \frac{V}{\pi r^2}$$

Wzór na pole powierzchni całkowitej (funkcja celu):

$$P = 2\pi r^2 + 2\pi r h$$

Po podstawieniu h do wzoru na pole, otrzymujemy funkcję jednej zmiennej r:

$$P(r) = 2\pi r^2 + \frac{2V}{r}$$

Do znalezienia minimum funkcji zastosowano numeryczną **metodę Newtona-Raphsona**. Warunkiem koniecznym istnienia ekstremum jest zerowanie się pierwszej pochodnej funkcji celu ($P(r) = 0$).

Wymagane pochodne funkcji celu:

- Pierwsza pochodna: $P'(r) = 4\pi r - \frac{2V}{r^2}$
- Druga pochodna: $P''(r) = 4\pi - \frac{4V}{r^3}$

Wzór iteracyjny metody Newtona dla poszukiwania miejsca zerowego pochodnej:

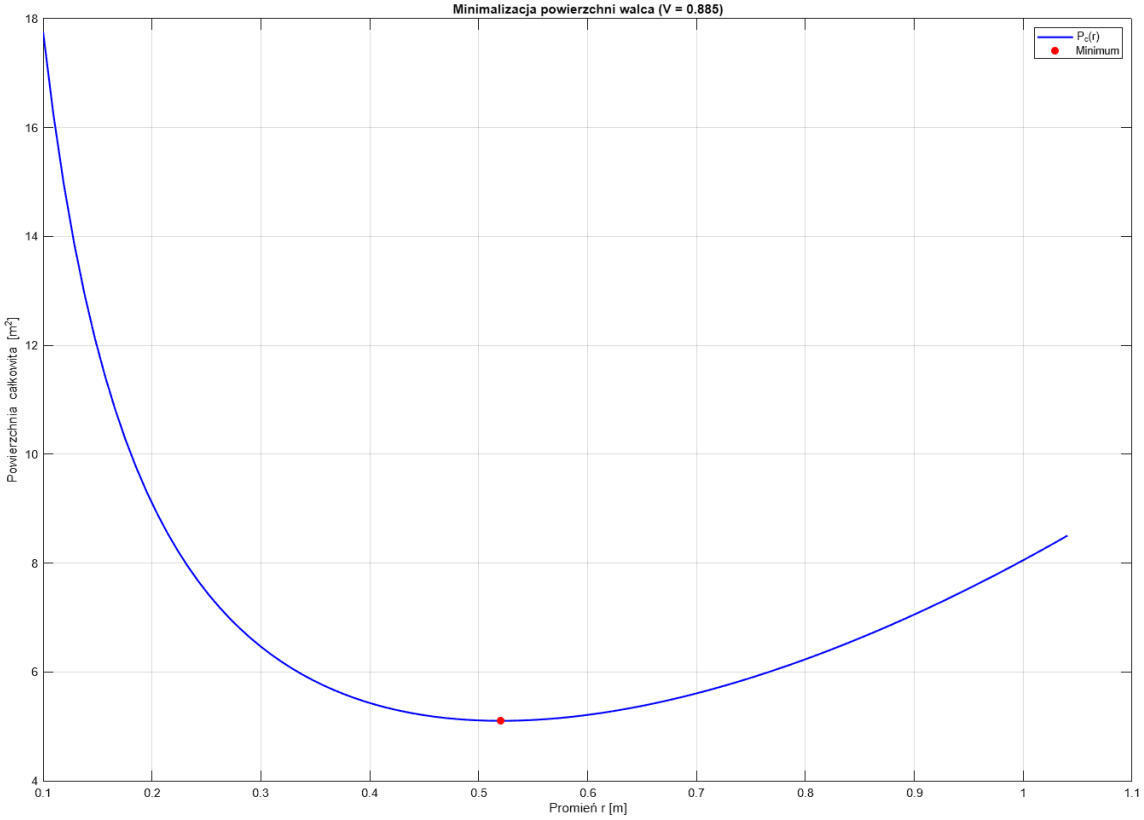
$$r_{n+1} = r_n - \frac{P'(r_n)}{P''(r_n)}$$

Kod matlaba:

```
fprintf('Zadanie: Minimalizacja powierzchni walca przy stałej objętości V =
%.6f\n', x_opt_vpasolve);
TargetFunc = @(r) 2*pi*r.^2 + 2*x_opt_vpasolve./r;
dTarget = @(r) 4*pi*r - 2*x_opt_vpasolve./(r.^2);
ddTarget = @(r) 4*pi + 4*x_opt_vpasolve./(r.^3);

fileID = fopen('wyniki_newton.txt', 'w');
fprintf(fileID, 'Iteracja\t Promien_r\t Pole_P(r)\t Pierwsza_Pochodna\n');
fprintf('Iter\t r_approx\t P(r)\t\t P''(r)\n');
fprintf('-----------------------------------------------------\n');

r_current = 1.0;
iter = 0;

while true
    iter = iter + 1;

    f_val = TargetFunc(r_current);
    df_val = dTarget(r_current);
    ddf_val = ddTarget(r_current);

    fprintf(fileID, '%d\t %.8f\t %.8f\t %.8f\n', iter, r_current, f_val, df_val);
    fprintf('%d\t %.6f\t %.6f\t %.8f\n', iter, r_current, f_val, df_val);

    r_next = r_current - (df_val / ddf_val);

    if abs(r_next - r_current) < epsilon || iter >= max_iter
        r_opt = r_next;
        break;
    end
```

```
    r_current = r_next;
end

fclose(fileID);

h_opt = x_opt_vpasolve / (pi * r_opt^2);
S_opt = TargetFunc(r_opt);

fprintf('-----------------------------------------------------\n');
fprintf('\nWYNIKI KOŃCOWE:\n');
fprintf('Plik z historią iteracji: wyniki_newton.txt\n');
fprintf('Optymalny promień (r):    %.6f [m]\n', r_opt);
fprintf('Optymalna wysokość (h):   %.6f [m]\n', h_opt);
fprintf('Minimalna powierzchnia:   %.6f [m^2]\n', S_opt);
fprintf('Liczba iteracji:          %d\n', iter);

fprintf('Stosunek h/r:             %.4f\n', h_opt/r_opt);
```

```
Zadanie: Minimalizacja powierzchni walca przy stałej objętości V = 0.885000
Iter    r_approx     P(r)         P'(r)
-----------------------------------------------------
1       1.000000     8.053185     10.79637069
2       0.329683     6.051716     -12.14177306
3       0.438719     5.243827     -3.68294997
4       0.506310     5.106575     -0.54216359
5       0.519918     5.102820     -0.01444480
6       0.520300     5.102818     -0.00001063
-----------------------------------------------------

WYNIKI KOŃCOWE:
Plik z historią iteracji: wyniki_newton.txt
Optymalny promień (r):    0.520301 [m]
Optymalna wysokość (h):   1.040602 [m]
Minimalna powierzchnia:   5.102818 [m^2]
Liczba iteracji:          6
Stosunek h/r:             2.0000
```



Poniższy wykres prezentuje przebieg zmienności pola powierzchni w funkcji promienia. Czerwonym punktem zaznaczono znalezione minimum globalne. Na podstawie wyznaczonego promienia optymalnego obliczono wysokość walca oraz zweryfikowano poprawność rozwiązania, sprawdzając geometryczną zależność dla optymalnego walca $(\mathrm{h} = 2\mathrm{r})$.